

Assignment QAA

2022-10-12

PA1454 - Quality Assurance Agency

Group 3: Johan Nilsson, Nikola Antic, Waled Mohammad, Tarek Alhoumsi



Introduction to chosen system

The chosen project to analyze for this assignment will be Numpy. Numpy (Numerical Python) is a library that is used for example when working with arrays, linear algebra, fourier transforms and matrices [1]. Three quality attributes will be targeted in the analysis and those will be **modifiability**, **testability** and **usability**. The analysis tool we will be working with will be codescene, introduced in the first workshop.

Current system state

Code health

The current state of the code health in Numpy is average with a score of 5.40 according to codescene. As seen in figure (1) there are quite a lot of hotspots with bad code health (red dots) and declining code health (yellow dots).

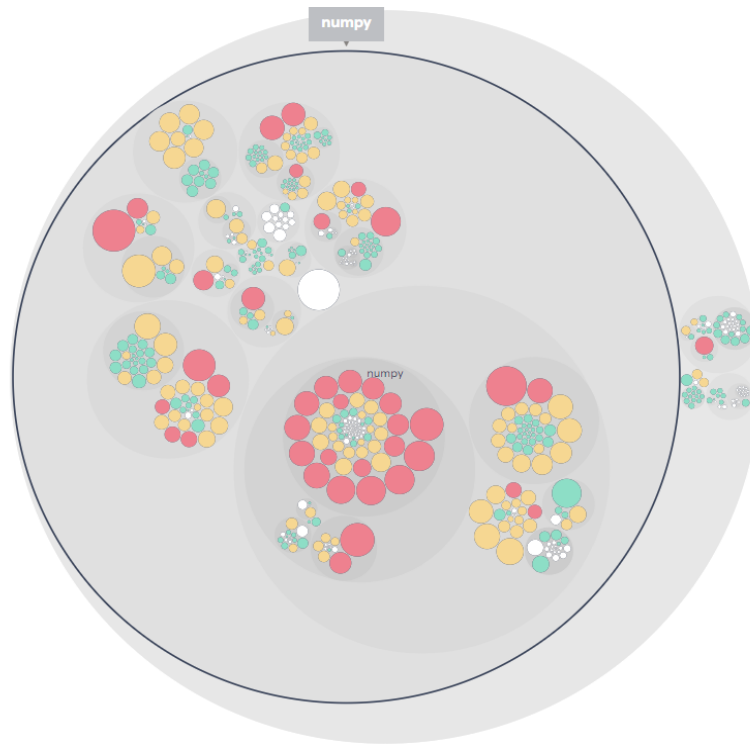
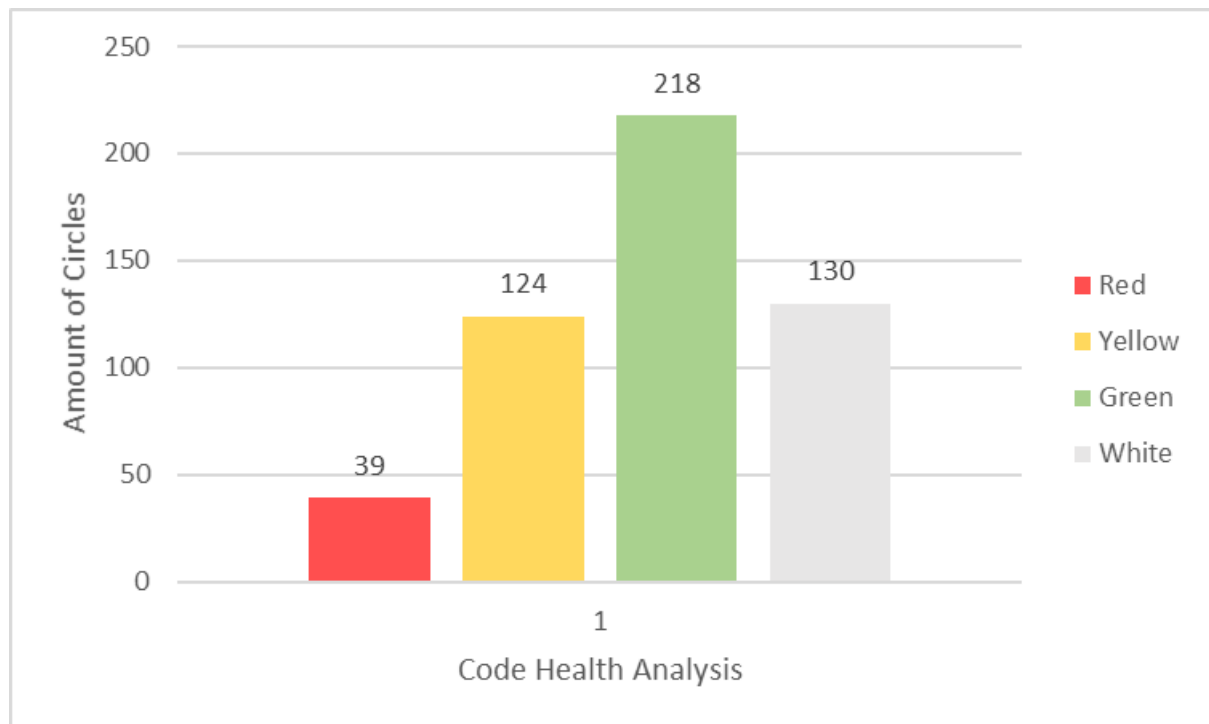
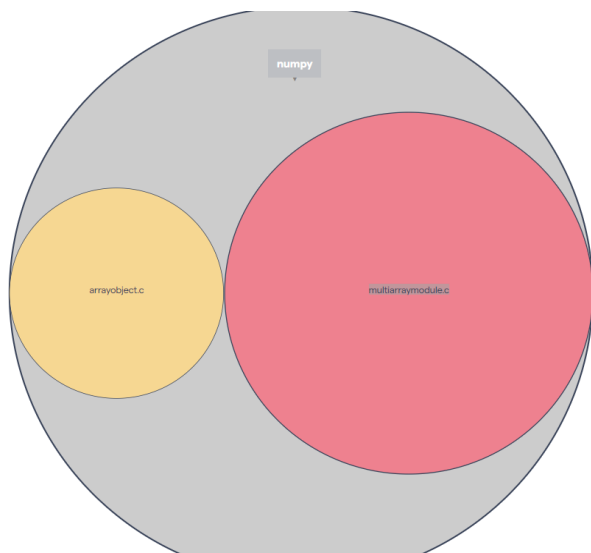


Figure 1: Code health in numpy.



The yellow dots indicate an increased maintenance cost and an increased risk for defects, meanwhile the red dots have surpassed the yellow stage and now represent technical debt. With further investigation we can see that the module `multiarray` has the worst code health of them all, see figure (3). If we narrow it down even more the technical debt traced to one file and that is the “`multiarraymodule.c`” file as seen in figure (4).



As mentioned above the red and yellow dots affect the modifiability of the project as it gets more expensive to maintain the system and risks arise more often. The technical depth is also something that affects the modifiability, because technical depth is cheap solutions made instead of doing the “proper” solutions and that affects changes in the future, making them more costly and time consuming. As for testability there are some modules that are not really supporting testability but overall the testability of Numpy is healthy as seen in the picture below (5).

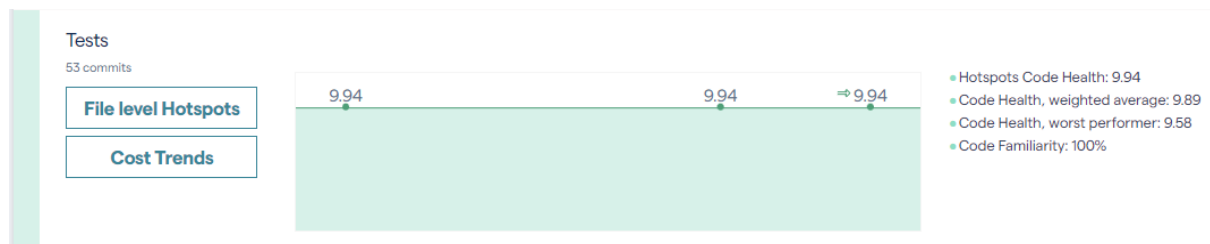


Figure 5: Tests in Numpy

The Usability of Numpy is not directly affected by code health or the coupling but it may be indirectly affected by the modifiability as changes may lead to downtime.

Coupling

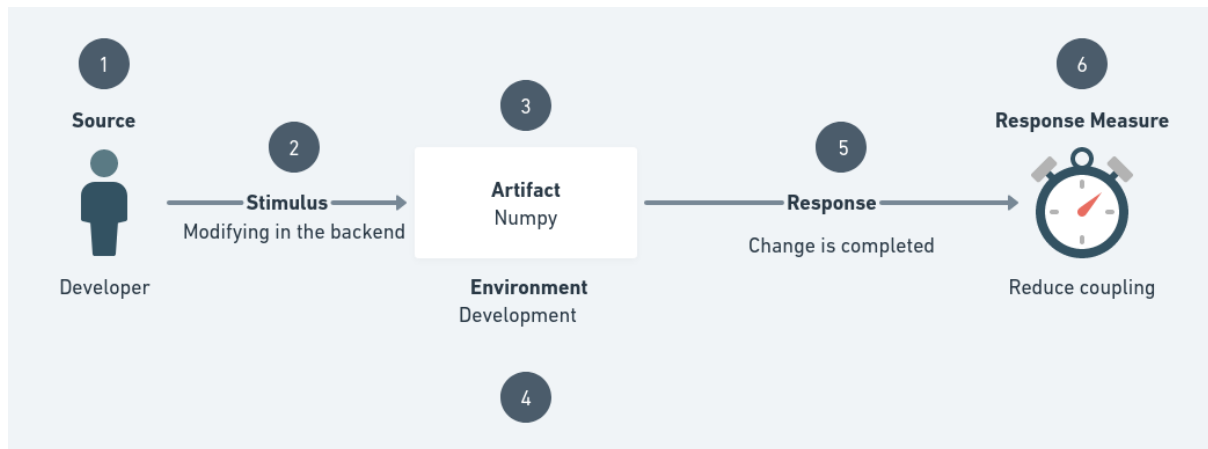
The coupling in Numpy is quite alright, the system is neither tightly or loosely coupled. It's leaning more towards loosely coupled. Every file is generally at maximum coupled to five others. This is good for modifiability because when a change is made in one place, there are not a lot of other places that are affected in the system. The testability can be affected both in the negative and positive way. If tests are run locally in some part of the system, low coupling doesn't really affect the tests as much as if you wanted to test a bigger part of the system with components that are not coupled together, meaning that testing will be more time consuming because the tested files are not connected. Again the usability is not really affected by the coupling of Numpy.

Why did we select usability?

We decided to pick from a list of the most common Quality Attributes to assure that it is viable to provide a rationale and tests. The main reason behind this choice is from personal experience while using numpy, and we came to the conclusion that usability is one of the main reasons why we have used numpy in the past. The most important thing with systems like numpy is to make it usable, since it exists for making it easier for the user. If it suddenly was very hard and complicated to use, many users would simply stop using it because it does not provide the simplicity that users want from this kind of system.

Modifiability

General scenario



Measurement

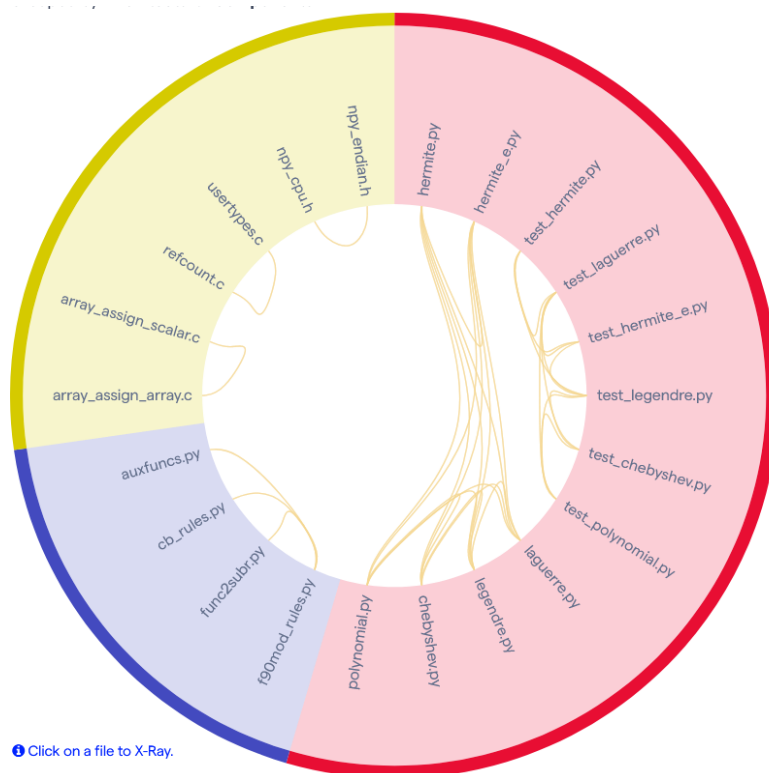


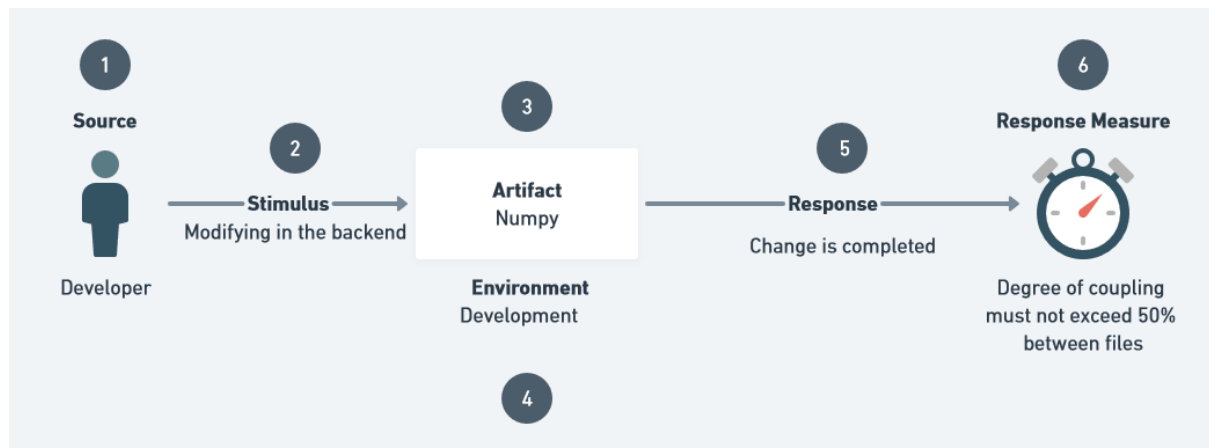
Figure 6 Change coupling in Numpy, the overlay is architectural components (minimum degree of coupling: 50%)

As you see in figure 6 there are three colors, each color is an architectural component and has their own files. No files from a specific architectural component are paired to another architectural component and that is positive.

However there is high coupling between files in the same architectural components, for example between file *func2subr.py* and *f90mod_rules.py* the degree of coupling is 60% and that is quite high when the two files are in the same architectural component.

To fix this problem we need to raise the level of abstraction and redesign the architectural components so as to reduce the coupling between the files. This decision can be translated into development by calling the architects that have designed the component and discussing the coupling with them and seeing if any restrictions are needed.

Concrete scenario



Measurement

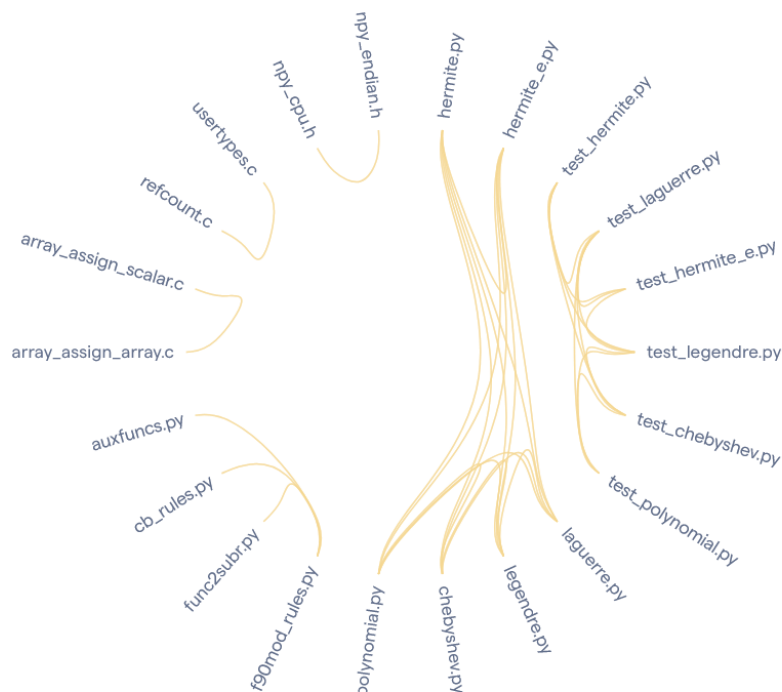


Figure 7 Change coupling in Numpy (minimum degree of coupling: 50%)

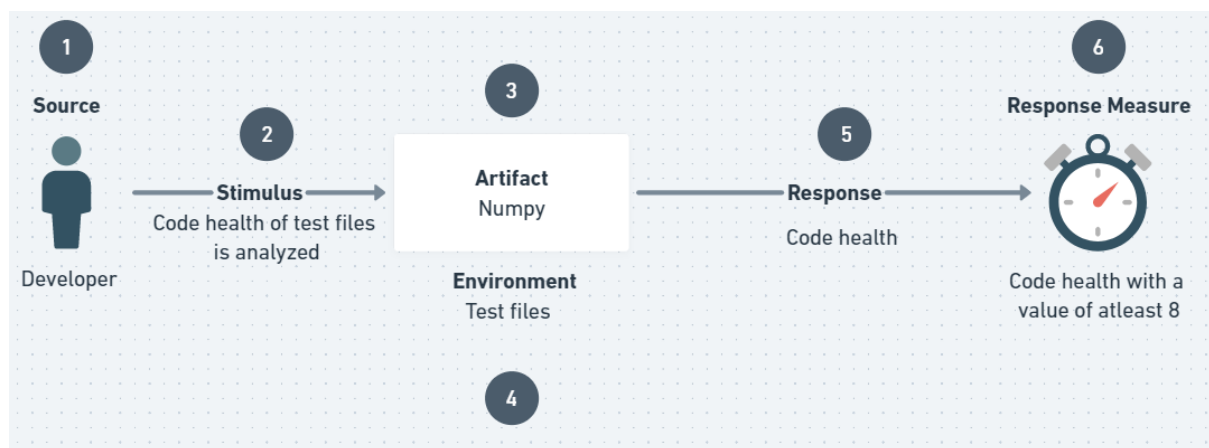
As you can see in figure (7), when the degree of coupling is minimum 50% then there are several files with 50 % degree of coupling or more. When the degree of coupling is exceeding 50% then the files are being dependent on other files that are handling other stuff in the library.

For example we can see in figure 7 we can see that file *hermite_e.py* has a coupling with file *laguerre.py* and the degree of coupling between them is 87%, that is not sustainable the files are dependent on each other. So if a change will be made on one of these files then the other file will be affected and need also be changed.

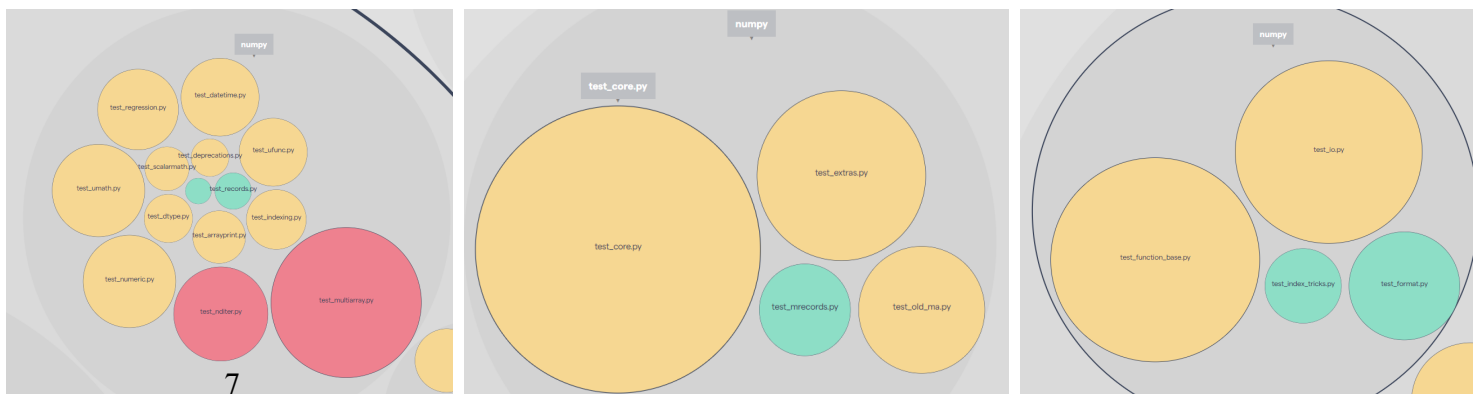
One way to decrease this high coupling between files is to work with functional design, which seeks to limit the responsibilities of modules along functionality between the files. Another way to reduce the high coupling between the files is to encapsulate the related fields and methods together. Those decisions can be translated into development by having a group that is responsible to work with the functional design. After the functional design is done then another group will work to encapsulate the fields and methods in every file.

Testability

General scenario



Measurement General scenario

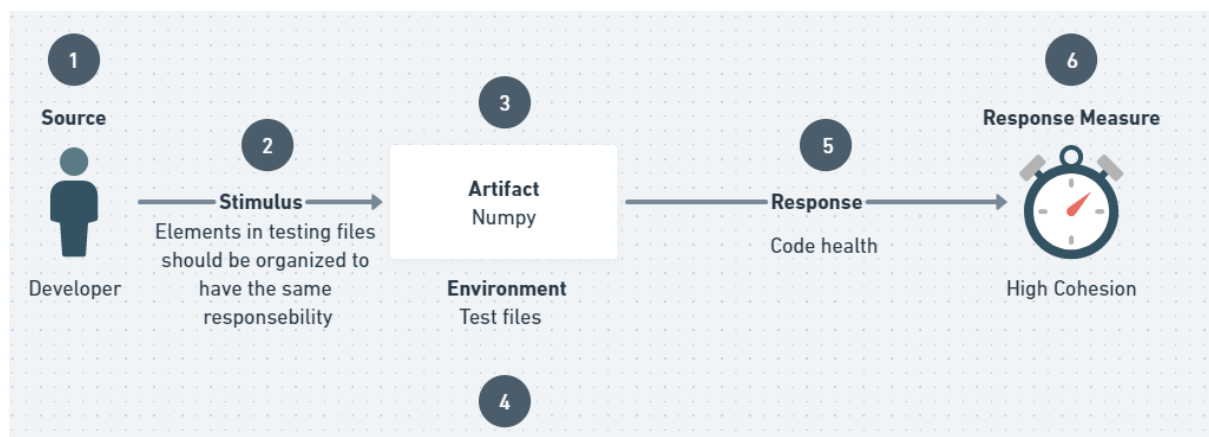


These are all the main test files in NumPy, there are just a few others but they show the same results. As we can see there are not a lot of green dots means that the code health of the test files are in general not that great. This depends on more than one factor, as seen in the picture below there are some examples.

- Low Cohesion
- Modularity Issue
- Bumpy Road
- Deep, Nested Complexity

In general these have all occurred as a consequence of lazy code writing, which is not good as it may lead to technical debt. To fix this the code has to be properly structured and organized and written in the “correct” way. A better written will make it easier to understand the system, maintain the system, increase code health and of course decrease technical debt.

Concrete scenario



Measurement Concrete scenario

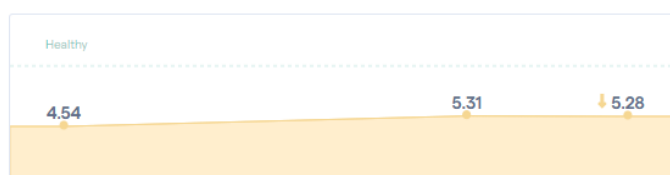
As of now the code health of the test files are in general not extremely well made as seen in the picture below (this is not all test files). All of the test files have a declining code health with a low cohesion.

test_umath.py
numpy/numpy/core/tests/

Plan a goal

Xray

Code Review



- Low Cohesion
 - Modularity Issue
 - Bumpy Road
 - Deep, Nested Complexity
- [Read More](#)

[View Code](#)

test_core.py
numpy/numpy/core/tests/

- Low Cohesion
- Modularity Issue

A big part of the decline in code health is to some extent because of low cohesion. Cohesion is a measure of how well the elements in a file belong together. With low cohesion elements in a file do not relate to each other and do not serve the same responsibilities. Below it can be seen how the difference in element responsibility is in one of the files (this applies to more files) is bad as it makes the code harder to understand, maintain and test.

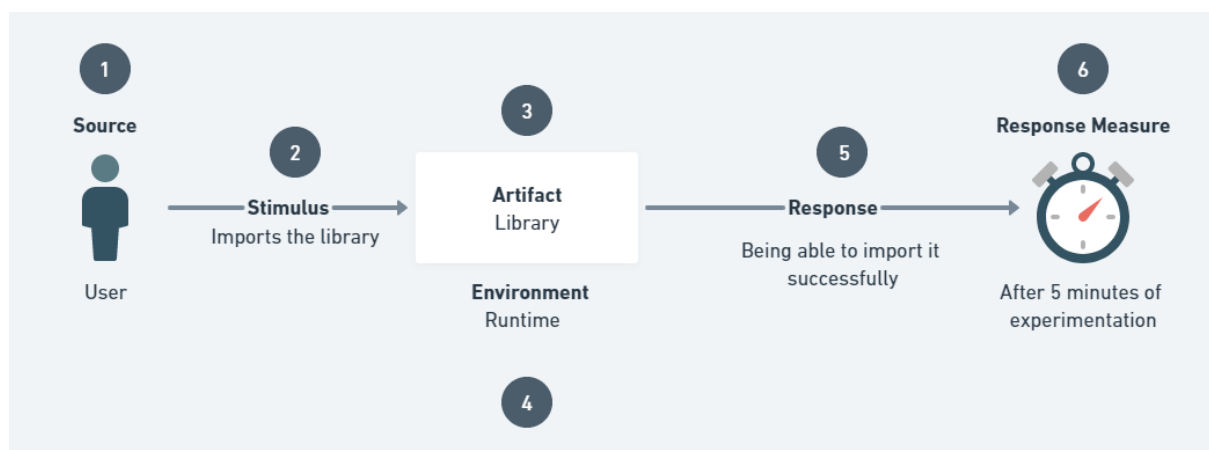
How to fix it?

The module has more than 10 different responsibilities amongst its 241 functions.

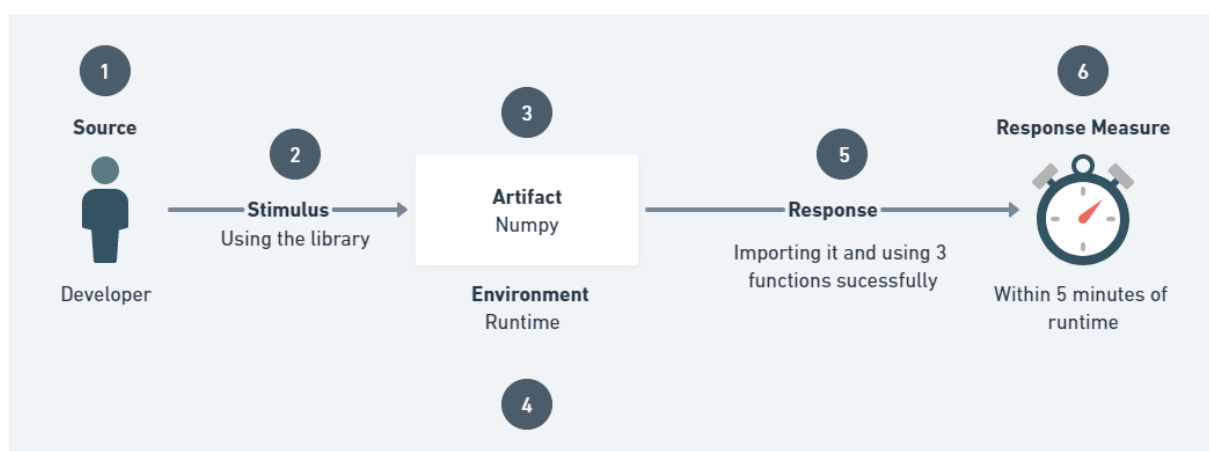
A development decision has to be made to increase cohesion and that is done by dividing the already existing files into smaller test files, each corresponding to their own responsibility. Functions that belong together should still be located together.

Usability

General scenario



Concrete scenario



Measurement

Measuring Usability is not as easy as measuring the other Quality Attributes, since usability can't be tested through code. Therefore all group members will do the same test to determine how good the usability is for numpy.

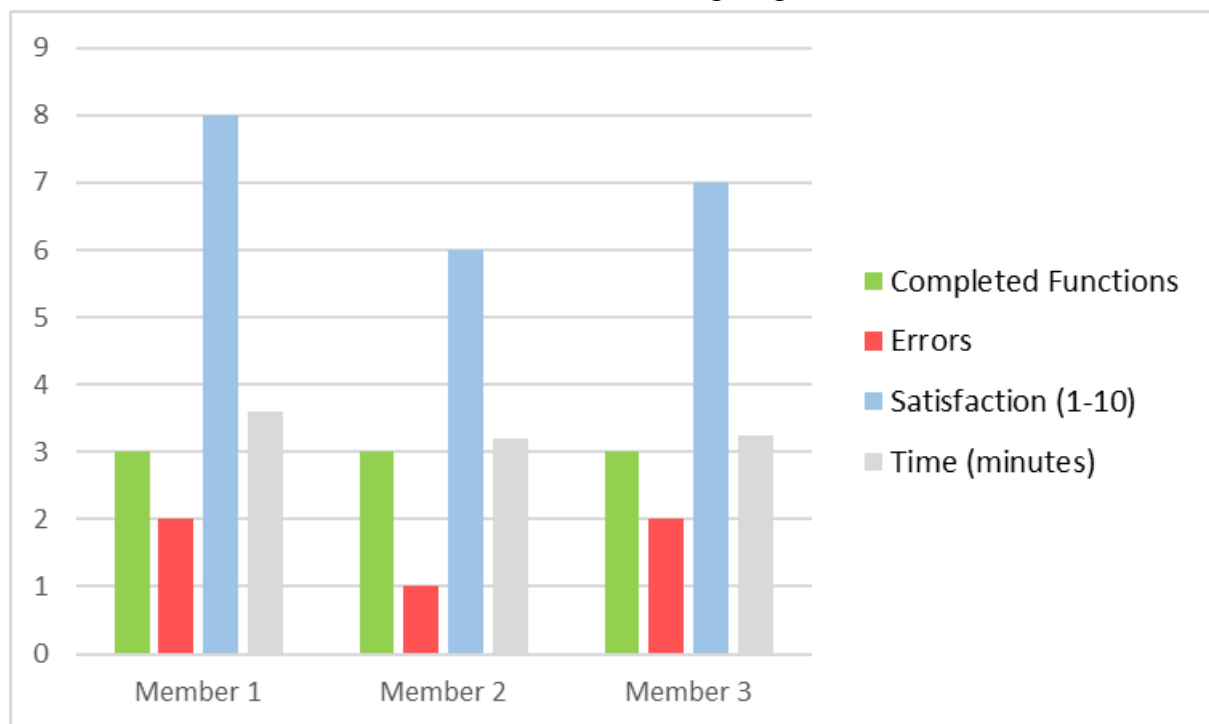
The test will be done by every group member receiving a code template with 3 different functions. The group members then have to enter a valid function from numpy to make the function work as intended for all three functions. After completing the test, each member will answer the following questions:

1. How many functions did you complete?
2. How many errors did you make?
3. How satisfied are you with numpy 1-10?
4. How many minutes did the test take?

The code template will look like this:

```
1  import numpy as np
2
3  #Function 1 - Create an array with the the values 1,2,3,4,5
4  array1 = #Enter function here
5  print("Array:" ,array1)
6
7  #Function 2 - Create an array with every even number between 1 and 10 using a function
8  array2 = #Enter function here
9  print("Array:", array2)
10
11 #Function 3 - Create an array with 10 random numbers between 1 and 10 using a function
12 array3 = #Enter function here
13 print("Array:", array3)
```

The results from the test in summarized in the following diagram:



As we can see in the diagram, all of the group members which participated in the tests manage to complete all of the functions within 5 minutes of runtime, which means that the Concrete Scenario passed.

Even though the Scenario was fulfilled, there still were 5 errors between all of the tests which is something that could be improved. In this case, we have to keep in mind that human error is behind some of these errors.

A potential improvement for numpy when it comes to Usability could be a better description and user guide for all of the functions that are provided in the library to help minimize the potential for human errors. For example, what the functions do, how to enter the variables in the parameter and what each of them does.

References

[1] W3schools (2022). NumPy Introduction.

https://www.w3schools.com/python/numpy/numpy_intro.asp [2022-10-05]